COSC 120: Introduction to Python Programming

In-Class Midterm 2 Examination

Prof. Jordan C. Hanson

Name: _____ **Student ID:** [ ]

Instructions: Choose the best answer for each question. Fill in the circle next to exactly one answer, A–D. Each question is worth 4 points.

Answer Sheet

1 ◯A ◯B ◯C ◯D	**2** ◯A ◯B ◯C ◯D	**3** ◯A ◯B ◯C ◯D	**4** ◯A ◯B ◯C ◯D	**5** ◯A ◯B ◯C ◯D
6 ◯A ◯B ◯C ◯D	**7** ◯A ◯B ◯C ◯D	**8** ◯A ◯B ◯C ◯D	**9** ◯A ◯B ◯C ◯D	**10** ◯A ◯B ◯C ◯D
11 ◯A ◯B ◯C ◯D	**12** ◯A ◯B ◯C ◯D	**13** ◯A ◯B ◯C ◯D	**14** ◯A ◯B ◯C ◯D	**15** ◯A ◯B ◯C ◯D
16 ◯A ◯B ◯C ◯D	**17** ◯A ◯B ◯C ◯D	**18** ◯A ◯B ◯C ◯D	**19** ◯A ◯B ◯C ◯D	**20** ◯A ◯B ◯C ◯D
21 ◯A ◯B ◯C ◯D	**22** ◯A ◯B ◯C ◯D	**23** ◯A ◯B ◯C ◯D	**24** ◯A ◯B ◯C ◯D	**25** ◯A ◯B ◯C ◯D

Dictionaries and Comprehensions

1. Consider the dictionary

```
students = {
    2601001: "Ana",
    2601002: "Ben",
    2601003: "Chloe"
}
```

What is produced by

```
students[2601002]
```

◯ **A.** Ana

◯ **B.** Ben

◯ **C.** 2601002

◯ **D.** An error occurs

2. Suppose the key `2601999` is not in `students`. What is returned by

```
students.get(2601999)
```

◯ **A.** 0

◯ **B.** False

◯ **C.** None

◯ **D.** The program always stops with an error

3. Consider

```
names = ["Ana", "Ben", "Chloe",
         "Diego", "Elena"]
first_id = 2601001
students = {
    first_id + i: name
    for i, name in enumerate(names)
}
```

Which key is associated with `"Diego"`?

◯ **A.** 2601001

◯ **B.** 2601002

◯ **C.** 2601003

◯ **D.** 2601004

4. Which statement best describes the difference between these two expressions?

```
[first_id + i
 for i in range(len(names))]

{first_id + i: name
 for i, name in enumerate(names)}
```

◯ **A.** The first creates a list of IDs; the second creates ID–name pairs in a dictionary

◯ **B.** Both create dictionaries

◯ **C.** The first creates a dictionary; the second creates a list

◯ **D.** Both create exactly the same object

5. In our lookup-timing exercise, why did we use

```
time.perf_counter()
```

◯ **A.** To sort the student records

◯ **B.** To measure short execution times with a high-resolution timer

◯ **C.** To count how many dictionary keys exist

◯ **D.** To pause execution for a fixed number of seconds

6. A student ID is near the end of a nested list containing N records. Which statement best describes our timing experiment as N becomes large?

◯ **A.** Both nested-list and dictionary lookup times must grow in exactly the same way

◯ **B.** Dictionary lookup becomes impossible once $N > 1000$

◯ **C.** Nested-list lookup generally grows with N, while dictionary lookup changes much less

◯ **D.** Nested-list lookup becomes constant-time because the list is nested

Classes and Objects

7. In a class definition, what is the main purpose of

```
def __init__(self, ...):
```

◯ **A.** To initialize a new object's attributes

◯ **B.** To delete an object

◯ **C.** To import another module

◯ **D.** To begin a recursive call

8. Inside an instance method, what does `self` refer to?

◯ **A.** The class file on disk

◯ **B.** Every object that has ever been created

◯ **C.** The Python interpreter

◯ **D.** The particular object on which the method was called

9. Consider

```
p1 = Particle2D(1, 2, 3, -1)
p1.update(0.5)
```

where `update()` uses

```
self.x += self.vx * dt
self.y += self.vy * dt
```

What are the new values of `p1.x` and `p1.y`?

◯ **A.** `(4, 1)`

◯ **B.** `(2.5, 1.5)`

◯ **C.** `(1.5, 2.5)`

◯ **D.** `(3, -1)`

10. Suppose

```
p1 = Particle2D(1, 2, 3, -1)
p2 = Particle2D(-2, 0, 1, 2)
p1.update(0.5)
```

What happens to `p2`?

◯ **A.** Its position changes by the same amount as `p1`

◯ **B.** Its velocity becomes equal to `p1`'s velocity

◯ **C.** It remains unchanged because each object has its own state

◯ **D.** Python deletes `p2`

11. A particle has velocity components

$$(v_x, v_y) = (3, 4).$$

What should its `speed()` method return?

◯ **A.** 5

◯ **B.** 7

◯ **C.** 12

◯ **D.** 25

12. Let

```
z1 = ComplexNumber(3, 2)
z2 = ComplexNumber(5, -4)
z3 = z1 + z2
```

If `__add__()` correctly implements complex-number addition, what are `z3.real` and `z3.imag` in the Chapter 11 version of the class?

◯ **A.** `(15, -8)`

◯ **B.** `(-2, 6)`

◯ **C.** `(8, 6)`

◯ **D.** `(8, -2)`

13. Let

$$z_1 = 3 + 2i, \qquad z_2 = 5 - 4i.$$

If `__mul__()` correctly implements complex-number multiplication, what is $z_1 z_2$?

◯ **A.** $7 + 6i$

◯ **B.** $23 - 2i$

◯ **C.** $15 - 8i$

◯ **D.** $8 - 2i$

Recursion and Binary Search

14. Which statement best defines a **base case** in a recursive function?

◯ **A.** The first line of the function

◯ **B.** The case that makes the largest recursive call

◯ **C.** A stopping case that returns without making another recursive call

◯ **D.** A loop that always runs at least once

15. Consider the recursive pattern from class:

```
def all_positive(values):
    if len(values) == 0:
        return True
    x = values.pop()
    if x <= 0:
        return False
    return all_positive(values)
```

What is returned for

```
all_positive([6, 2, -9, 4])
```

◯ **A.** False

◯ **B.** True

◯ **C.** -9

◯ **D.** None

16. What is the value of

$$5!$$

◯ **A.** 25

◯ **B.** 60

◯ **C.** 100

◯ **D.** 120

17. Which expression is the recursive case for factorial?

◯ **A.** `factorial(n + 1)`

◯ **B.** `n * factorial(n - 1)`

◯ **C.** `n + factorial(n)`

◯ **D.** `factorial(0) * n`

18. Use the sorted list

```
values = [3, 7, 11, 18, 23,
          29, 34, 42, 47, 53,
          61, 68, 74, 83, 91]
```

A recursive binary search is called for target `74`. What index should ultimately be returned?

◯ **A.** 7

◯ **B.** 11

◯ **C.** 12

◯ **D.** 13

19. In our recursive binary-search function, what does

```
if low > high:
    return -1
```

mean?

◯ **A.** There are no indices left to search, so the target is absent

◯ **B.** The target must be at index 0

◯ **C.** The list should be reversed

◯ **D.** The target has been found at `mid`

20. Why is binary search efficient on a large sorted list?

◯ **A.** It always examines every element exactly once

◯ **B.** It converts the list into a dictionary first

◯ **C.** It works only because recursive functions are always constant-time

◯ **D.** Each decision discards about half of the remaining search region

Inheritance and Vector2D

21. A `Vector2D` object has components `x = 3` and `y = 4`. What should

```
v.magnitude()
```

return?

◯ **A.** 7

◯ **B.** 5

◯ **C.** 12

◯ **D.** 25

22. Which method correctly computes the angle of a two-dimensional vector in the class used in our warm-up?

◯ **A.** `math.sqrt(self.x**2 + self.y**2)`

◯ **B.** `math.atan2(self.x, self.y)`

◯ **C.** `math.atan2(self.y, self.x)`

◯ **D.** `self.x + self.y`

23. Consider

```
class ComplexNumber(Vector2D):
    ...
```

Which statement is correct?

◯ **A.** `ComplexNumber` is a child class that inherits from `Vector2D`

◯ **B.** `Vector2D` inherits from `ComplexNumber`

◯ **C.** The two classes are unrelated

◯ **D.** Python combines the two classes into a dictionary

24. In

```
def __init__(self, real, imag):
    super().__init__(real, imag)
```

what does `super().__init__()` do?

◯ **A.** It creates an unrelated `Vector2D` object and discards it

◯ **B.** It calls the recursive base case

◯ **C.** It overrides every method in the parent class

◯ **D.** It reuses the parent class constructor to initialize the inherited components

25. Why did we override `__add__()` inside the inherited `ComplexNumber` class?

◯ **A.** Because inherited methods can never perform arithmetic

◯ **B.** Because the parent method returns a `Vector2D`, while complex addition should return a `ComplexNumber`

◯ **C.** Because `Vector2D` has no `x` or `y` attributes

◯ **D.** Because overriding prevents `magnitude()` from being inherited

End of Exam

Check that exactly one answer is clearly marked for each question.